

Rules vs Skills in Cursor

Cursor's AI can be customised in two complementary ways: Rules that set standing context and Skills that trigger specific procedures. Understanding the difference makes your AI assistant dramatically more consistent and useful.

The One-Line Distinction

Rules tell the AI **how to always behave** in your project. **Skills** tell the AI **what to do** when you ask it to perform a specific task.

Think of it this way: Rules are like your project's standing orders — they run quietly in the background on every conversation. Skills are like saved recipes — you invoke them by name when you want a specific outcome.

What Are Rules?

A Rule is a Markdown file (`.mdc`) that Cursor injects into the AI's context. Rules carry project knowledge, coding standards, architectural decisions, and constraints. There are four trigger types:

Trigger Type	Typical location	When it's active
Always	<code>.cursor/rules/*.mdc</code>	Every single conversation — no exceptions
Auto Attached	<code>.cursor/rules/*.mdc + glob</code>	Only when editing files that match a glob pattern (e.g. *.py)
Agent Requested	<code>.cursor/skills//SKILL.md</code>	The AI decides to pull it in based on the task at hand
Manual	<code>.cursor/skills//SKILL.md</code>	Only when you type @skill-name in the chat

Examples of Rules

Rules carry context that should always influence how the AI writes code:

Always Rule — injected into every conversation

```
# .cursor/rules/project-standards.mdc
```

```
---
```

description: Core project coding standards

alwaysApply: true

```
---
```

```
# Project Standards
```

This is a Python 3.12 FastAPI project using SQLAlchemy (async) and Pydantic v2.

```
## Coding Rules
```

- Use type hints on every function — no bare `dict` or `list`.
- All route handlers must have docstrings.
- Never use mutable default arguments (e.g. `def f(x=[])`). Use `None` instead.
- Raise `HTTPException` with a clear `detail` string, not bare exceptions.
- API responses must always use a Pydantic response model.

Auto Attached Rule — only active when editing test files

```
# .cursor/rules/test-standards.mdc
```

```
---
```

description: Testing conventions for `test_*.py` and `*_test.py` files

globs: `['**/test_*.py', '**/*_test.py']`

```
---
```

```
# Testing Standards
```

Use `pytest` with `pytest-asyncio` for async tests.

- Test one behaviour per test function.
- Name tests: `test_<function>_<scenario>` (e.g. `test_create_user_duplicate_email`).
- Use `pytest` fixtures for DB setup/teardown — never share state between tests.
- Mock external HTTP calls with `respx` (for `httpx`) or `responses` (for `requests`).

What Are Skills?

A Skill is a rule set to **Manual** or **Agent Requested** trigger that encapsulates a step-by-step procedure for a specific task. Rather than describing how the AI should always behave, a Skill describes *what it should do* when you invoke it by name.

Skills are **verbs** ("do X"). Rules are **adjectives** ("always be Y"). A Rule says "always add type hints and docstrings." A Skill says "when I type @write-unit-tests, generate pytest coverage for the selected function."

Examples of Skills

Each Skill is a Manual rule you invoke on demand with @**skill-name**:

Manual Skill — invoked with @write-unit-tests | .cursor/skills/write-unit-tests/SKILL.md

.cursor/skills/write-unit-tests/SKILL.md

Skill: Write Unit Tests

When invoked, perform these steps:

1. Identify all inputs, outputs, and side effects of the selected function.
2. List edge cases: empty input, None, boundary values, exception paths.
3. Write pytest tests — one test function per scenario.
4. Use pytest.mark.asyncio for async functions.
5. Mock external calls with unittest.mock.patch or pytest-mock's mocker fixture.
6. Aim for at least 80% branch coverage.

Manual Skill — invoked with @create-api-endpoint | .cursor/skills/create-api-endpoint/SKILL.md

.cursor/skills/create-api-endpoint/SKILL.md

Skill: Create API Endpoint

When invoked, scaffold a FastAPI route that includes:

1. A Pydantic request model and a Pydantic response model.
2. Input validation handled automatically by FastAPI/Pydantic.
3. HTTPException raised with appropriate status codes (400, 404, 500).
4. An async route function with a docstring describing the endpoint.
5. A corresponding pytest-asyncio test using httpx.AsyncClient.

Key Differences at a Glance

	Rule	Skill
Purpose	Define how the AI always behaves	Define what the AI does for a specific task
Trigger	Always / File glob / Auto	Manual (@name) or Agent Requested
Runs	Automatically — every time	Only when you ask for it
Content	Standards, context, constraints	Step-by-step procedure / checklist
Analogy	Company style guide	Standard operating procedure (SOP)
Token cost	Paid on every prompt	Paid only when invoked
Example invocation	(automatic)	@write-unit-tests

When to Use Rules vs Skills

Use a Rule when...

- The AI needs to **know something on every single prompt** — e.g. your tech stack, folder structure, team conventions.
- You want to **prevent recurring mistakes** — e.g. the AI keeps using **bare except:** instead of catching specific exception types.
- The context is **scoped to a file type** — e.g. "when editing SQL files, always use parameterised queries."
- You need to **describe the project architecture** so the AI doesn't suggest patterns that don't fit.

Use a Skill when...

- You have a **repeatable task** you run frequently — writing pytest tests, scaffolding a new API endpoint, adding error handling.
- The task has a **specific multi-step procedure** you want to enforce consistently every time.
- You want to **save tokens** — don't inject a 200-line checklist on every prompt if you only need it once a day.
- You want to **share a workflow with your team** — commit the **.cursor/rules/** folder and everyone gets the same Skill.

Quick Decision Guide

Question	If YES → use
Should this apply on every single AI interaction?	Rule (Always)
Does it only matter for specific file types?	Rule (Auto Attached)
Is it a step-by-step task I invoke by name?	Skill (Manual)
Should the AI decide when to pull it in?	Skill (Agent Requested)
Does it define standards rather than actions?	Rule
Does it describe a procedure or checklist?	Skill

Side-by-Side: Real Project Example

Imagine you're building a Python FastAPI REST API. Here's how you'd split responsibilities between Rules and Skills:

Rules (.cursor/rules/*.mdc)

- ✓ Stack: Python 3.12, FastAPI, SQLAlchemy async, Pydantic v2
- ✓ Folder structure: /routers, /models, /schemas, /services
- ✓ Always add type hints. Never use bare `except:`.
- ✓ Auto-attached for `test_*.py` → use `pytest + pytest-asyncio` (.cursor/rules/)
- ✓ Auto-attached for `*.sql` → always use parameterised queries

Skills (.cursor/skills/<name>/SKILL.md)

- @write-unit-tests → generate `pytest` tests for selected function
- @create-api-endpoint → scaffold FastAPI route + Pydantic models
- @add-error-handling → wrap in `try/except` with `HTTPException`
- @create-migration → generate Alembic migration for schema change
- @optimize-query → review SQLAlchemy query, suggest eager loading

How to Create Rules and Skills in Cursor

Step 1 — Create the folder structure

```
mkdir -p .cursor/rules # for always-on Rules (.mdc files)
mkdir -p .cursor/skills/write-unit-tests # one subfolder per Skill
mkdir -p .cursor/skills/add-endpoint
# Or use Cursor's UI: Settings → Rules → + New Rule (for rules)
# Agent Panel → Skills → + New Skill (for skills)
```

Step 2 — Create a Rule (.mdc file in .cursor/rules/)

```
Always Rule — .cursor/rules/project-context.mdc
# File: .cursor/rules/project-context.mdc
---
description: Always-on project context
alwaysApply: true
---

This is a Python 3.12 FastAPI project with PostgreSQL (asyncpg driver).
Use SQLAlchemy 2.x async sessions. All models are in app/models/.
Always use type hints. Always raise HTTPException, never bare exceptions.
```

Step 3 — Create a Skill (SKILL.md in .cursor/skills/)

Manual Skill — .cursor/skills/add-endpoint/SKILL.md

File: .cursor/skills/add-endpoint/SKILL.md

Skill: Add Endpoint

When I type @add-endpoint, do the following:

1. Ask for the resource name and HTTP method if not provided.
2. Create a Pydantic request model and a Pydantic response model.
3. Write an async route function with docstring in app/routers/{resource}.py.
4. Raise HTTPException(400) for bad input, HTTPException(404) for not found.
5. Register the router in app/main.py.
6. Generate a pytest-asyncio test using httpx.AsyncClient.

Step 4 — Invoke the Skill in Cursor Chat

In Cursor's chat panel:

@add-endpoint Create a POST /users/register endpoint

Cursor pulls in the skill's procedure and executes every step.

Pro Tips

Keep Rules short and factual. Long, vague rules dilute the AI's attention. Each Always rule should ideally be under 30 lines.

Name Skills as verbs. @skill-write-tests is clearer than @tests. The name should say exactly what the AI will do.

Commit the entire .cursor/ folder to git. Both .cursor/rules/ (Rules) and .cursor/skills/ (Skills) are then shared with your whole team automatically — no per-machine setup needed.

Move a Rule to a Skill if it's rarely needed. If a rule is rarely relevant, convert it to a Manual Skill to save tokens and reduce noise in the AI's context.